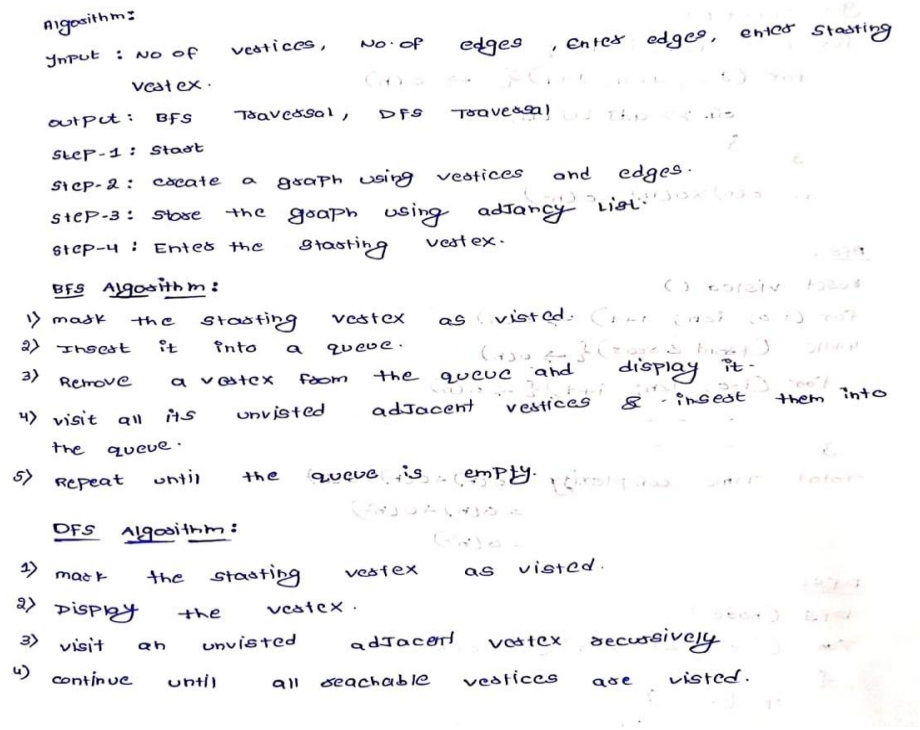
Practical –8

Aim: Implementation of graph and search (BFS & DFS)

Algorithm:



Algorithm:
Input : No of vertices, No. of edges, Enter edges, enter starting vertex.
output: BFS Traversal, DFS Traversal
step-1 : start
step-2 : create a graph using vertices and edges.
step-3 : store the graph using adjancy list.
step-4 : Enter the starting vertex.

BFS Algorithm:
1) mark the starting vertex as visted.
2) Insert it into a queue.
3) Remove a vertex from the queue and display it.
4) visit all its unvisted adjacent vertices & insert them into the queue.
5) Repeat until the queue is empty.

DFS Algorithm:
1) mark the starting vertex as visted.
2) Display the vertex.
3) visit an unvisted adjacent vertex recursively
4) continue until all reachable vertices are visted.

Program:

```
#include&lt; iostream&gt;
using namespace std;

class Graph {

public:
   int adj [10][10];
   int visited [10];
```

```
int n;

// ── Get Input ──
void getInput() {
    cout &lt;&lt; &quot;Enter number of vertices  : &quot;;
    cin  &gt;&gt; n;

    cout &lt;&lt; &quot;Enter adjacency matrix (&quot; &lt;&lt; n &lt;&lt;&quot;x&quot; &lt;&lt; n &lt;&lt; &quot;)  :&quot; &lt;&lt; endl;
    for (int i = 0; i &lt; n; i++)
        for (int j = 0; j &lt; n; j++)
            cin &gt;&gt; adj[i][j];
}

// ── Reset Visited ──
void resetVisited() {
    for (int i = 0; i &lt; n; i++)
        visited[i] = 0;
}


void BFS(int start) {
    int queue[10];
    int front = 0, rear = 0;

    resetVisited();
```

```
    // enqueue start vertex
    queue[rear++] = start;
    visited[start] = 1;

    cout &lt;&lt; &quot;BFS Traversal   : &quot;;

    While (front &lt; rear) {

        // dequeue
        int node = queue[front++];
        cout &lt;&lt; node &lt;&lt; &quot; &quot;;

        // visit all unvisited neighbours
        For (int i = 0; i &lt; n; i++) {
            if(adj[node][i] == 1 &amp; &amp; visited[i] == 0) {
                queue[rear++] = i;
                visited[i]    = 1;
            }
        }
    }
    cout &lt;&lt; endl;
}
```

```
void DFS(int node) {
    visited[node] = 1;
    cout &lt;&lt; node &lt;&lt; &quot; &quot;;

    for(int i = 0; i &lt; n; i++) {
        if(adj[node][i] == 1 &amp;&amp; visited[i] == 0) {
            DFS(i);   // recursive call
        }
    }
}

// ── Start DFS ──
void startDFS(int start) {
    resetVisited();
    cout &lt;&lt; &quot;DFS Traversal : &quot;;
    DFS (start);
    cout &lt;&lt; endl;
}

// ── Display Adjacency Matrix ──
void displayMatrix() {
    cout &lt;&lt; &quot;\nAdjacency Matrix :&quot; &lt;&lt; endl;
    cout &lt;&lt; &quot;  &quot;;
    For (int i = 0; i &lt; n; i++)
        cout &lt;&lt; i &lt;&lt; &quot; &quot;;
```

```
        cout &lt;&lt; endl;

        for(int i = 0; i &lt; n; i++) {
            cout &lt;&lt; i &lt;&lt; &quot; &quot;;


            for(int j = 0; j &lt; n; j++)
                cout &lt;&lt; adj[i][j] &lt;&lt; &quot; &quot;;
            cout &lt;&lt; endl;

        }
    }
};

// ── MAIN ──
int main () {

    Graph g  ;   // object

    g.getInput();
    g.displayMatrix();

    int start;
    cout &lt;&lt; &quot;\nEnter starting vertex   : &quot;;
    cin  &gt;&gt; start;
```

```
    cout &lt;&lt; endl;
    g.BFS(start);
    g.startDFS(start);

    return 0;
}
```

Output:

```
Enter number of vertices: 5
Enter number of edges: 4
Enter edges (u v):
0 1
0 2
1 3
2 4
Enter starting vertex: 0

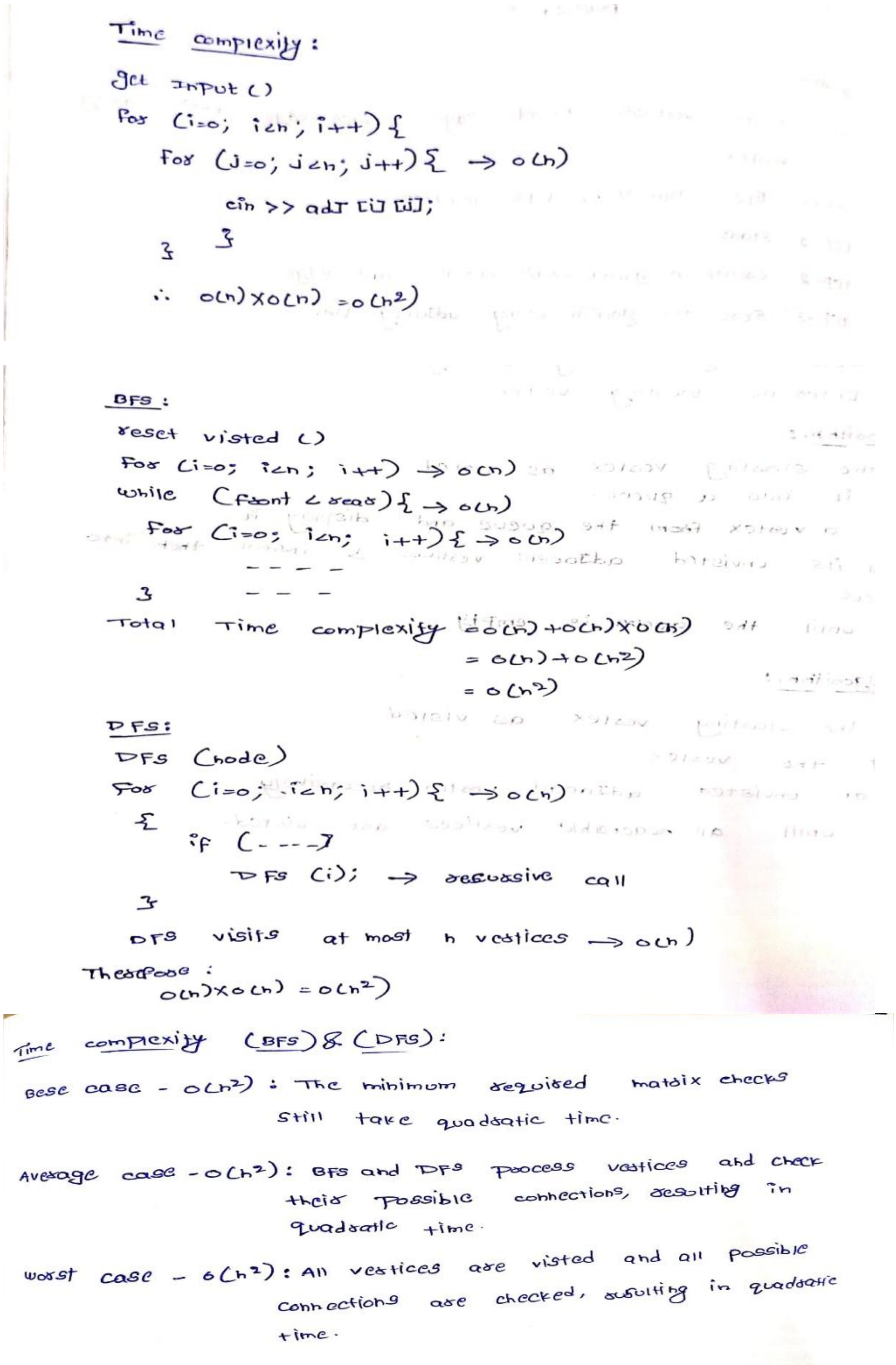
DFS Traversal: 0 1 3 2 4

BFS Traversal: 0 1 2 3 4

Execution Time:
DFS: 2593200 ns
BFS: 727500 ns
```

Name:M.Ravi Kiran Reddy
Enrollment Number:92460118230

Time complexity :



Conclusion : Thus, the graph was successfully implemented using an adjacency matrix, and BFS and DFS traversals were performed successfully. The traversal methods helped in understanding how vertices of a graph can be visited systematically.

Name:M.Ravi Kiran Reddy
Enrollment Number:92460118230